АСЭ
РОСАТОМ

Лекция 1

Vite, Vite+, Oxc и Nginx

Эрнандес Гарсия Даниэль Дельевич

Веб-разработчик

О чём лекция

Vite — dev-сервер и сборщик для современных фронтенд-проектов.

Vite+ — единый CLI для зависимостей, проверок, тестов и сборки.

Oxc — быстрые инструменты для JS/TS на Rust: Oxlint и Oxfmt.

Nginx — раздача статики, SPA-fallback, proxy и HTTPS.

Как устроены инструменты, как их настраивать и как собрать проект.

РАЗДЕЛ

01

Vite

Dev-сервер, конфиг и сборка

Создание проекта

Новый проект создаётся командой npm create vite@latest.

В существующий проект Vite добавляют как dev-зависимость: npm install -D vite.

Стартовые шаблоны: react, react-ts, vue, svelte, solid, preact, lit, vanilla.

Vite — это dev-сервер и сборщик: в разработке отдаёт ESM по запросу, в сборке собирает бандл.

vite.config.ts

Файл конфига TS-проекта — vite.config.ts.

Минимальный конфиг: import { defineConfig } from 'vite'; export default defineConfig({}).

Если конфиг зависит от команды, экспортируется функция: defineConfig(({ command }) => ({ ... })).

command = serve при запуске vite, command = build при vite build.

.env не читаются через process.env: Vite откладывает их загрузку, потому что набор файлов зависит от конфига.

В process.env внутри конфига доступны только переменные окружения ОС.

process.env в Node.js — глобальный объект с переменными окружения процесса.

Вручную .env загружает loadEnv: const env = loadEnv(mode, process.cwd(), '').

process.cwd() — текущий рабочий каталог процесса Node.js.

root и base

root — папка, в которой лежит index.html. По умолчанию — process.cwd(), каталог запуска команды.

base — префикс для деплоя в подпапку, а не в корень домена. По умолчанию '/'.

В сборке base подставляет префикс во все пути и задаёт import.meta.env.BASE_URL.

В dev-сервере приложение поднимается по этому пути: localhost:5173/app/.

define

АСЭ
РОСАТОМ

define заменяет идентификаторы в коде на этапе сборки — аналог #define в C.

В dev-режиме добавляет свойство в globalThis.

В сборке меняет код на уровне AST, подставляя значение вместо идентификатора.

Строковые значения подставляются как есть: строки нужно оборачивать в JSON.stringify.

Глобалы из define типизируют в vite-env.d.ts.

```
define: { __APP_VERSION__: JSON.stringify('1.0.0') }
```

plugins и publicDir

plugins — массив плагинов. false, null и undefined игнорируются: удобно для isDev && plugin().

Вложенные массивы распаковываются, Promise дожидается разрешения перед запуском.

publicDir — папка статичных ассетов, которые отдаются как есть (favicon и т.п.). По умолчанию public.

В dev-режиме файлы доступны по корню /, в сборке копируются в корень outDir (dist).

publicDir: false отключает обслуживание и копирование папки.

resolve

АСЭ
РОСАТОМ

resolve.extensions позволяет опускать расширения в import. По умолчанию: .mjs, .js, .mts, .ts, .jsx, .tsx, .json.

Если подходит несколько файлов, побеждает расширение, стоящее в списке раньше.

resolve.tsconfigPaths включает алиасы из compilerOptions.paths в tsconfig.json. По умолчанию false.

clearScreen, envDir, envPrefix

АСЭ
РОСАТОМ

clearScreen очищает терминал при логах и пересборках. По умолчанию true.

Отключают, чтобы не терять историю логов при HMR.

envDir — папка, из которой грузятся .env. По умолчанию root; false отключает загрузку.

envPrefix — префикс переменных, доступных в клиенте через import.meta.env. По умолчанию VITE_.

Меняют, чтобы защитить серверные секреты от попадания в клиентский бандл.

Dev-сервер

server.port по умолчанию 5173.

strictPort запрещает занимать другой порт: если указанный занят, сервер не запустится.

https включает TLS/HTTP2 для dev-сервера; принимает { key, cert } или плагин @vitejs/plugin-basic-ssl.

open открывает приложение в браузере при старте; строковое значение — путь, который нужно открыть.

cors настраивает CORS dev-сервера: true — любой origin, или объект настроек как CORS-middleware.

proxy перенаправляет запросы dev-сервера на бэкенд, обходя CORS.

forwardConsole перенаправляет события консоли браузера в терминал Vite; включается true.

Полезен при разработке с AI-агентами — им становятся видны ошибки клиента.

Плагины и ассеты

АСЭ
РОСАТОМ

basicSsl() подключается в plugins:

Плагин для React — @vitejs/plugin-react.

Предварительно упакованные зависимости кэшируются в node_modules/.vite.

import imgUrl from './img.png' при сборке превращается в URL файла.

imgUrl используется как обычный URL, например в src тега <img>.

```
import basicSsl from '@vitejs/plugin-basic-ssl';
export default { plugins: [basicSsl()] };
```

.env и режимы

В клиентский код попадают переменные с префиксом VITE_.

Все значения в import.meta.env — строки, даже числа и булевы.

Сравнение с булевым — явное: import.meta.env.VITE_F === 'true'.

Без префикса VITE_ переменная вернёт undefined.

Приоритет загрузки: .env.[mode].local → .env.[mode] → .env.local → .env.

MODE — текущий режим (development или production).

BASE_URL — базовый URL приложения из опции base.

PROD — true в продакшен-сборке, DEV — true в разработке, SSR — true при серверном рендеринге.

Кастомные переменные — строки, встроенные PROD/DEV/SSR — boolean.

import.meta.env и типы

import.meta — объект с метаданными текущего ESM-модуля.

import.meta.env — объект с переменными окружения, который создаёт Vite.

ImportMetaEnv — интерфейс, описывающий структуру import.meta.env.

TypeScript по умолчанию знает только MODE, DEV, PROD, BASE_URL, SSR.

Обращение к import.meta.env.VITE_APP_TITLE без типизации даёт ошибку типа.

Кастомные переменные типизируют в src/vite-env.d.ts.

```
interface ImportMetaEnv {
  readonly VITE_APP_TITLE: string
}
interface ImportMeta {
  readonly env: ImportMetaEnv
}
```

Директива /// <reference types="vite/client" /> подключает типы встроенных import.meta.env и ассетов.

Препроцессоры



АСЭ
РОСАТОМ

Для .scss, .sass, .less и .styl плагины не нужны — поддержка встроена.

Нужно установить сам препроцессор как dev-зависимость: npm install -D sass.

Vite учитывает алиасы из своего конфига в @import для Sass/Less.

АСЭ
РОСАТОМ

РАЗДЕЛ

02

Vite+

Единый CLI для проекта

Что такое Vite+

Vite+ состоит из глобального CLI vp и локального пакета vite-plus в каждом проекте.

Конфигурация — в стандартном vite.config.ts.

Запуск dev-сервера — vp dev.

Сборка — vp build.

Обновление глобального CLI — vp upgrade.

Для CI — voidzero-dev/setup-vp.

Зависимости

АСЭ
РОСАТОМ

vp add <pkg> — добавить пакет.

vp add -D <pkg> — добавить dev-пакет.

vp remove <pkg> — удалить пакет.

vp update — обновить зависимости.

Менеджер пакетов задаётся полем devEngines.packageManager в package.json.

Точная версия — полем packageManager, например bun@1.1.20.

Проверки: check и lint

vp check объединяет форматирование, линтинг и проверку типов.

vp check --fix включает автоисправление.

vp lint запускает линтинг, vp lint --fix — с автоисправлением.

typeCheck запускает компилятор TypeScript во время линтинга.

typeAware ищет логические ошибки на основе типов.

vp lint основан на Oxlint.

Конфиг lint — в vite.config.ts, в блоке lint; не в oxlint.config.ts.

```
export default defineConfig({
  lint: { ignorePatterns: ['dist/**'] },
});
```

Форматирование и тесты



АСЭ
РОСАТОМ

vp fmt форматирует код.

vp fmt . --write записывает результат в файлы.

vp fmt основан на Oxfmt.

Конфиг fmt — в vite.config.ts, в блоке fmt; не в oxfmt.config.ts.

vp test запускает тесты.

Тесты основаны на Vitest.

Конфиг test — в vite.config.ts, в блоке test; не в vitest.config.ts.

Скрипты и задачи

АСЭ
РОСАТОМ

vp run <script-name> запускает скрипт из package.json.

vp dev — встроенная команда, vp run dev запускает скрипт dev из package.json.

Задачи описываются в vite.config.ts, в блоке run.tasks.

Зависимость одной задачи от другой — поле dependsOn.

Запуск задачи — vp run <task-name>.

Задача — именованный скрипт, состояние которого Vite+ кеширует по входным и выходным файлам.

vpx — аналог npx/bunx в Vite+.

```
run: {
  tasks: {
    build: { command: 'vp build', dependsOn: ['lint'] },
  },
}
```

Git-хуки

АСЭ
РОСАТОМ

vp hooks enable включает хуки, vp hooks disable выключает.

vp hooks status показывает статус.

Скрипты лежат в .vite-hooks в корне проекта; папку нужно коммитить, чтобы хуки были одинаковыми у всех.

pre-commit настраивается файлом .vite-hooks/pre-commit с командой vp staged.

vp staged проверяет только изменённые файлы перед коммитом.

Через staged в vite.config: staged: { '*.{js,ts,tsx}': 'vp check --fix' }.

РАЗДЕЛ

03

Oxc

Oxlint и Oxfmt на Rust

Oxc и Oxlint

Oxc — набор высокопроизводительных инструментов для JavaScript и TypeScript на Rust.

Oxlint — высокопроизводительный линтер для JS/TS на стеке Oxc, совместимый с ESLint.

Oxlint использует tsgo, он же TypeScript 7.

Установка: npm install -D oxlint.

npm-скрипты: "lint": "oxlint", "lint:fix": "oxlint --fix".

Конфиг Oxlint

Инициализация конфига — oxlint --init.

Файл конфига — .oxlintrc.json или oxlint.config.ts.

Базовый конфиг:

```
import { defineConfig } from 'oxlint';
export default defineConfig({});
```

Плагины Oxlint

Плагин — именованная группа правил; его включение делает правила доступными.

Встроенные плагины включаются в поле plugins, например plugins: ['import'].

Запись plugins переопределяет плагины, включённые по умолчанию.

Плагин eslint включён по умолчанию и подключает основные правила ESLint.

АСЭ
РОСАТОМ

РАЗДЕЛ

04

Nginx

Раздача SPA, proxy и HTTPS

Зачем Nginx фронтенду

Статика: HTML, JS, CSS и шрифты — быстрее и дешевле, чем Node-сервер.

Reverse proxy: прячет API, снимает CORS и балансирует нагрузку между инстансами.

TLS и кэш: терминирует HTTPS, сжимает gzip и кэширует ассеты на клиенте.

В деплое Vite-приложения Nginx — точка входа, а Node/API — за ним.

Карта конфига

main — user, worker_processes, error_log.

events — worker_connections.

http — mime.types, gzip, upstream.

server — listen, server_name, root.

location — правила по URI, try_files, proxy_pass.

Простые директивы заканчиваются ;, блочные — { }.

include /etc/nginx/conf.d/*.conf; подключает файлы.

Настройки наследуются сверху вниз: http → server → location.

Основной файл — /etc/nginx/nginx.conf.

Что настроить в первую очередь



АСЭ
РОСАТОМ

listen и server_name — порт и домен.

root и index — корень статики и главный файл.

location — правила по URI.

try_files — SPA-fallback на index.html.

proxy_pass — запросы в API.

upstream — несколько бэкендов.

expires и gzip — кэш и сжатие.

ssl и return 301 — HTTPS и редирект.

Минимальный конфиг для SPA

АСЭ
РОСАТОМ

```
server {
  listen 80;
  server_name example.com;
  root /usr/share/nginx/html;
  index index.html;

  location / {
    try_files $uri $uri/ /index.html;
  }

  location /api/ {
    proxy_pass http://backend:8080/;
  }
}
```

Без try_files прямой заход на /posts/1 даст 404. gzip, кэш и HTTPS добавляются в этот же server.

Проверка: nginx -t → nginx -s reload.

Итог

Vite — быстрый dev-сервер и сборщик; конфиг в vite.config.ts.

Vite+ — единый CLI vp для зависимостей, проверок, тестов и сборки.

Oxc — Oxlint и Oxfmt на Rust для быстрого линтинга и форматирования.

Nginx — точка входа в проде: статика, SPA-fallback, proxy и HTTPS.

Дальше: собрать проект на Vite+, настроить vp check и развернуть его за Nginx.